



Cloud Computing Project

Prepared by :

ABAHAMID Soufiane
ENNOUR Nassim

Supervised by :

Pr. LACHAIZE Renaud
Pr. ROPARS Thomas

Academic Year 2025/2026

Table of Contents

1	Introduction	6
2	Base steps	7
2.1	Deploying the original application in GKE	7
2.1.1	GKE Cluster Creation	7
2.1.2	Deploying the app	7
2.1.3	Reconfiguring the application	10
2.2	Analyzing the provided configuration	12
2.3	Deploying the load generator on a local machine	13
2.4	Deploying the Load Generator Automatically in Google Cloud	17
3	Advanced steps	22
3.1	Monitoring the application and the infrastructure	22
3.1.1	Design Choices	22
3.1.2	Technical Choices	22
3.1.3	Implementation	23
3.2	Performance evaluation	28
3.2.1	Environment Preparation	28
3.2.2	Evaluation Plan and Experiment Design	31
3.2.3	Results Analysis	32
3.3	Canary Releases	46
3.3.1	Setup and Microservice Preparation	46
3.3.2	Traffic Splitting and Observability	48
3.3.3	Full Transition to v2	51

4	Bonus steps	54
4.1	Performance evaluation	54
	References	55

List of Figures

2.1	default cluster configuration	7
2.2	Application pods status	8
2.3	Application pods status	8
2.4	Pods status in Autopilot	9
2.5	Removing load generator	10
2.6	Reducing emailservice's CPU requests	11
2.7	Pods state after reconfiguration	11
2.8	Upload button	14
2.9	Frontend IP address	15
2.10	Generated load	15
2.11	CPU Usage	16
2.12	Memory Usage	16
2.13	Frontend logs	17
2.14	main.tf	19
2.15	Dockerfile problematic part	20
2.16	Frontend logs 2	21
2.17	Container running inside the VM	21
3.1	Prometheus Chats	23
3.2	kube-prometheus-stack successfully installed	24
3.3	kube-prometheus-stack pods	24
3.4	Node cpu seconds total	26
3.5	Container cpu usage seconds total	26
3.6	Nodes Resources Dashboard	27
3.7	Pods Resources Dashboard	27

3.8	Master's startup script	29
3.9	Worker's startup script	30
3.10	CPU Heatmap	32
3.11	CPU requests per workload	34
3.12	CPU usage of node 1	35
3.13	CPU usage of node 2	35
3.14	CPU usage of node 3	36
3.15	Memory requests per workload	37
3.16	Memory usage of node 1	38
3.17	Memory usage of node 2	39
3.18	Memory usage of node 3	39
3.19	Receive bandwidth for each microservice	40
3.20	Failure rate vs users	42
3.21	Latency vs users	43
3.22	Throughput vs users	45
3.23	istio installed in the cluster	47
3.24	frontend v2 image build and push.	48
3.25	Cluster upscaled to 4 nodes	48
3.26	VirtualService manifest.	49
3.27	Traffic split between frontend v1 and v2 in Kiali	50
3.28	Unpredictable traffic split for currencyservice	51
3.29	Virtualservice manifest after shift to v2	51
3.30	Kiali traffic distribution after transition to v2	52
3.31	Traffic graph after v1 deletion	52
3.32	Frontend v2 served in browser	53

List of Abbreviations

GCP *Google Cloud Platform*

GKE *Google Kubernetes Engine*

VM *Virtual Machine*

CPU *Central Processing Unit*

IP *Internet Protocol*

YAML *YAML Ain't Markup Language*

gRPC *Google Remote Procedure Call*

HTTP *Hypertext Transfer Protocol*

UI *User Interface*

Chapter 1

Introduction

Microservice architectures are widely used in cloud computing to build scalable and resilient applications. Kubernetes has become the standard platform for deploying and managing such applications by providing powerful orchestration, scaling, and monitoring capabilities.

This lab assignment focuses on deploying and managing a microservice-based application on Google Cloud Platform (GCP) using Google Kubernetes Engine (GKE). The target application, Online Boutique, is a demo e-commerce platform developed by Google and composed of multiple interacting microservices. It serves as a realistic case study for exploring cloud-native application behavior.

The project is organized into base, advanced, and bonus steps. It covers application deployment, configuration analysis, load generation, monitoring, performance evaluation, and canary deployments. The goal is to gain practical experience with Kubernetes and cloud infrastructure while analyzing system behavior under different workloads and configurations.

Chapter 2

Base steps

2.1 Deploying the original application in GKE

2.1.1 GKE Cluster Creation

For this first step, we used the default GKE configuration. The following command was used to create the GKE cluster after setting the default zone to *europe-west6-a* :

```
gcloud container clusters create shopapp-cluster
```

We can observe that the cluster was created with the following default configuration (3 nodes, e2-medium machine type) :

```
nassimennour2@cloudshell:~ (cloud-k8s-project-478408)$ gcloud container clusters create shopapp-cluster
Note: Your Pod address range ('--cluster-ipv4-cidr') can accommodate at most 1008 node(s).
Creating cluster shopapp-cluster in europe-west6-a... Cluster is being health-checked (Kubernetes Control Plane is healthy)...done.
Created [https://container.googleapis.com/v1/projects/cloud-k8s-project-478408/zones/europe-west6-a/clusters/shopapp-cluster].
To inspect the contents of your cluster, go to: https://console.cloud.google.com/kubernetes/workload/_gcloud/europe-west6-a/shopapp-cluster?project=cloud-k8s-project-478408
kubeconfig entry generated for shopapp-cluster.
NAME: shopapp-cluster
LOCATION: europe-west6-a
MASTER_VERSION: 1.33.5-gke.1201000
MASTER_IP: 34.65.177.245
MACHINE_TYPE: e2-medium
NODE_VERSION: 1.33.5-gke.1201000
NUM_NODES: 3
STATUS: RUNNING
STACK_TYPE: IPV4
```

Figure 2.1: default cluster configuration

2.1.2 Deploying the app

In order to deploy the app, we followed the steps indicated in the project's README file:

- First, we cloned the project, then we applied the provided manifests:

```
git clone --depth 1 --branch v0 \
https://github.com/GoogleCloudPlatform/microservices-demo.git
cd microservices-demo/ && \
kubectl apply -f ./release/kubernetes-manifests.yaml
```

- Then, we checked the status of the pods:

```
kubectl get pods
```

We can observe that not all the pods are running, in particular, the **ad-service** and **load-generator** pods stay in the **Pending** status.

```
nassimennour2@cloudshell:~/microservices-demo (cloud-k8s-project-478408)$ kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
adservice-54fdcb4646-62jzj          0/1     Pending   0           5m5s
cartservice-7d76bb9df-7m2lb         1/1     Running   0           5m8s
checkoutservice-5d9d84cd44-pnvzp    1/1     Running   0           5m11s
currencyservice-569f6c566d-qjlf6    1/1     Running   0           5m7s
emailservice-7d4b8cd7d6-j8jq5       1/1     Running   1 (4m40s ago) 5m12s
frontend-76dbbdfc5-1bb7h            1/1     Running   0           5m10s
loadgenerator-56674fd696-pt8tp       0/1     Pending   0           5m7s
paymentservice-9ff6ffd6-4jz29        1/1     Running   0           5m10s
productcatalogservice-74c67b9d8b-7v8db 1/1     Running   0           5m9s
recommendationservice-5966b9f59d-bkwnk 1/1     Running   0           5m11s
redis-cart-c4fc658fb-gth5g           1/1     Running   0           5m8s
shippingservice-5565748dc4-6vb99      1/1     Running   0           5m6s
nassimennour2@cloudshell:~/microservices-demo (cloud-k8s-project-478408)$
```

Figure 2.2: Application pods status

The Pending status indicates that Kubernetes could not schedule these pods due to **insufficient resources** on the cluster nodes.

To investigate more, we ran the **kubectl describe** command on the load-generator pod :

```
Type      Reason          Age          From          Message
----      -
Warning    FailedScheduling 24m          default-scheduler 0/3 nodes are available: 3 Insufficient cpu.
coming pod.
Warning    FailedScheduling 13m (x2 over 18m) default-scheduler 0/3 nodes are available: 3 Insufficient cpu.
coming pod.
Normal     NotTriggerScaleUp 3m44s (x122 over 24m) cluster-autoscaler Pod didn't trigger scale-up:
nassimennour2@cloudshell:~/microservices-demo (cloud-k8s-project-478408)$
```

Figure 2.3: Application pods status

This confirms that the pod could not be scheduled due to **insufficient CPU resources** on all available nodes. No preemption or autoscaling

was triggered, highlighting the resource limitations of the default GKE cluster configuration.

2.1.2.1 Autopilot Mode Analysis

- What is GKE **Autopilot Mode**?

A fully managed mode where Google **automatically provisions and manages** the cluster infrastructure, including nodes and scaling.

- Why Does Autopilot Hide the Problem?

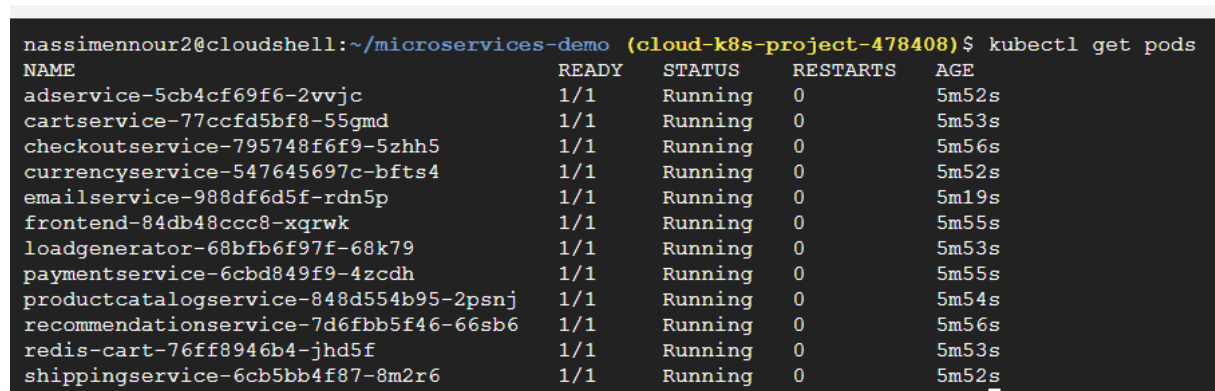
Because in Autopilot Mode, GKE ensures **enough resources** are available for all scheduled pods as defined in their requirements

- Testing the Autopilot Mode:

We created the cluster in autopilot mode using the following command :

```
gcloud container clusters create--auto shopapp--cluster--auto \
--region europe--west6
```

Then, we deployed the app using the same manifests as before and observed the state of the pods:



NAME	READY	STATUS	RESTARTS	AGE
adservice-5cb4cf69f6-2vvjc	1/1	Running	0	5m52s
cartservice-77ccfd5bf8-55gmd	1/1	Running	0	5m53s
checkoutservice-795748f6f9-5zhh5	1/1	Running	0	5m56s
currencyservice-547645697c-bfts4	1/1	Running	0	5m52s
emailservice-988df6d5f-rdn5p	1/1	Running	0	5m19s
frontend-84db48ccc8-xqrwk	1/1	Running	0	5m55s
loadgenerator-68bf6f97f-68k79	1/1	Running	0	5m53s
paymentservice-6cbd849f9-4zcdh	1/1	Running	0	5m55s
productcatalogservice-848d554b95-2psnj	1/1	Running	0	5m54s
recommendationservice-7d6fbb5f46-66sb6	1/1	Running	0	5m56s
redis-cart-76ff8946b4-jhd5f	1/1	Running	0	5m53s
shippingservice-6cb5bb4f87-8m2r6	1/1	Running	0	5m52s

Figure 2.4: Pods status in Autopilot

- Conclusion on Autopilot Mode:

We can observe that in Autopilot Mode, **all pods started successfully**,

showing that Autopilot automatically provisions enough resources. However, this mode is more expensive and less flexible, so **we chose to continue with standard mode** for this project.

2.1.3 Reconfiguring the application

To resolve the resource scheduling issue in the default GKE cluster, we customized the deployment using Kustomize:

1. Stopped deploying the load generator

We edited `kustomize/base/kustomization.yaml` and commented out the line referencing the loadgenerator in the resources section.

```
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
resources:
- adservice.yaml
- cartservice.yaml
- checkoutservice.yaml
- currencyservice.yaml
- emailservice.yaml
- frontend.yaml
# loadgenerator.yaml
- paymentservice.yaml
- productcatalogservice.yaml
- recommendationservice.yaml
- shippingservice.yaml
```

Figure 2.5: Removing load generator

2. Reduced Resource Requests for Less Critical Services

We identified **emailservice** and **recommendationservice** as less critical for the core shopping experience. So we divided by half their CPU **requests** parameter in their respective YAML files.

```

GNU nano 7.2                                     base/emailservice.yaml *
value: "1"
readinessProbe:
  periodSeconds: 5
  grpc:
    port: 8080
livenessProbe:
  periodSeconds: 5
  grpc:
    port: 8080
resources:
  requests:
    cpu: 50m
    memory: 64Mi
  limits:
    cpu: 200m
    memory: 128Mi

```

Figure 2.6: Reducing emailservice's CPU requests

Note: Kubernetes uses the *request*spending parameter to decide if a pod can be scheduled on a node, not *limits*.

3. Applied the Changes

The changes were applied using the following command :

```
kubectl apply -k kustomize/
```

As a result, we can observe that **all pods started successfully**, confirming that the resource issue was resolved.

```

nassimennour2@cloudshell:~/microservices-demo (cloud-k8s-project-478408)$ kubectl get pods

```

NAME	READY	STATUS	RESTARTS	AGE
adservice-54fdcb4646-zr2xx	1/1	Running	0	2m46s
cartservice-7d76bb9df-b8lv2	1/1	Running	0	2m46s
checkoutservice-5d9d84cd44-tt8s8	1/1	Running	0	2m46s
currencyservice-569f6c566d-wh2ts	1/1	Running	0	2m46s
emailservice-85586b9dc6-sgnkk	1/1	Running	0	2m45s
frontend-76dbbdfc5-pcnq6	1/1	Running	0	2m44s
paymentservice-9ff6fffd6-thbwj	1/1	Running	0	2m44s
productcatalogservice-74c67b9d8b-jkx8	1/1	Running	0	2m44s
recommendationservice-68b865bc9b-b9tts	1/1	Running	0	2m43s
redis-cart-c4fc658fb-h8x2g	1/1	Running	0	2m43s
shippingservice-5565748dc4-h5qnx	1/1	Running	0	2m43s

Figure 2.7: Pods state after reconfiguration

Note: The script used to reproduce this step can be found in our repository at:

base_steps/1_deploying_the_original_application_in_gke/scripts/deploying_the_original_application_in_gke.sh

2.2 Analyzing the provided configuration

We selected the **emailservice** microservice for analysis. Its configuration file defines three Kubernetes resources: **Deployment**, **Service**, and **ServiceAccount**.

- **Deployment:**

- **metadata**: Names and labels the deployment for identification.
- **selector** and **template.metadata.labels**: Ensure the deployment manages the correct pods.
- **serviceName**: Assigns a dedicated service account for security.
- **securityContext**: Enforces running as non-root and restricts filesystem access.
- **containers**: Specifies the container image, exposed port (8080), and environment variables (PORT, DISABLE_PROFILER).
- **readinessProbe** and **livenessProbe**: Use gRPC checks to monitor pod health.
- **resources**: Sets CPU and memory requests/limits to control resource usage.

- **Service:**

- **type**: Uses ClusterIP for internal cluster exposure.
- **selector**: Routes traffic to the correct pods.
- **ports**: Exposes port 5000 for gRPC, mapped to container port 8080.

- **ServiceAccount:**

- Provides pod identity and access control for the emailservice.

Overall, this configuration ensures that **emailservice** is securely deployed, resource limited, and reliably exposed within the cluster.

2.3 Deploying the load generator on a local machine

For this step, we chose to run the loadgenerator service in Google Cloud Shell instead of on our own computers.

The first thing we did was place all the loadgenerator files from `microservices-demo/src/loadgenerator` into a dedicated loadgenerator folder in Cloud Shell. To do this, we used the Google Cloud Console: we clicked on the three dots and then selected the Upload button:

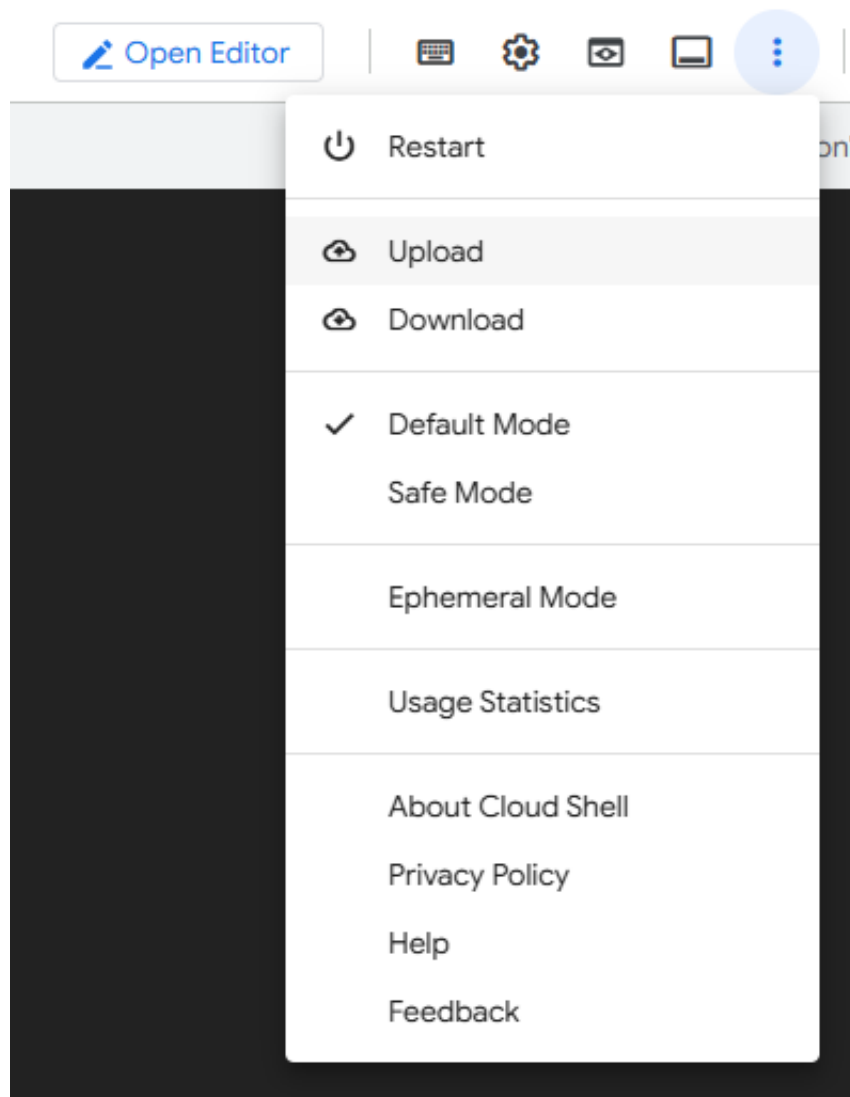


Figure 2.8: Upload button

Next, we searched for the IP address of the frontend load balancer. For this, we ran the command:

```
kubectl get services
```

```
soufiane@DESKTOP-RNOR9SM:~/workspace/cloud-project$ kubectl get services
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
adservice	ClusterIP	34.118.229.250	<none>	9555/TCP	74m
cartservice	ClusterIP	34.118.238.208	<none>	7070/TCP	74m
checkoutservice	ClusterIP	34.118.225.244	<none>	5050/TCP	74m
currencyservice	ClusterIP	34.118.236.18	<none>	7000/TCP	74m
emailservice	ClusterIP	34.118.233.116	<none>	5000/TCP	74m
frontend	ClusterIP	34.118.225.61	<none>	80/TCP	74m
frontend-external	LoadBalancer	34.118.225.240	34.65.106.168	80:32253/TCP	74m
kubernetes	ClusterIP	34.118.224.1	<none>	443/TCP	108m
paymentservice	ClusterIP	34.118.228.237	<none>	50051/TCP	74m
productcatalogservice	ClusterIP	34.118.232.171	<none>	3550/TCP	74m
recommendationservice	ClusterIP	34.118.236.145	<none>	8080/TCP	74m
redis-cart	ClusterIP	34.118.229.1	<none>	6379/TCP	74m
shippingservice	ClusterIP	34.118.238.139	<none>	50051/TCP	74m

Figure 2.9: Frontend IP address

Finally, we built the Docker image and ran it using the following commands:

```
docker build -t loadgenerator .
docker run --rm -e FRONTEND_ADDR=$FRONTEND_ADDR \
loadgenerator
```

After running the container, it displayed live statistics of the load being generated, showing some details such as request type, path and how many requests it sent and some other useful numbers, and they keep getting updated in real time:

Type	Name	# reqs	# fails	Avg	Min	Max	Med	req/s	failures/s
GET	/	20	0 (0.00%)	86	36	292	58	0.00	0.00
GET	/cart	24	0 (0.00%)	45	27	105	40	0.10	0.00
POST	/cart	35	0 (0.00%)	77	54	118	72	0.60	0.00
POST	/cart/checkout	10	0 (0.00%)	83	48	306	52	0.20	0.00
GET	/product/0P0K6V6EV0	16	0 (0.00%)	43	32	97	36	0.10	0.00
GET	/product/1Y0MWN1N40	18	0 (0.00%)	53	32	273	35	0.20	0.00
GET	/product/2Z1FJ3GM2N	19	0 (0.00%)	41	32	80	36	0.10	0.00
GET	/product/66VCNSJNHP	14	0 (0.00%)	52	32	108	41	0.10	0.00
GET	/product/6E922MYFZ	8	0 (0.00%)	48	35	86	40	0.00	0.00
GET	/product/9SIQT8TOJO	9	0 (0.00%)	65	32	273	37	0.30	0.00
GET	/product/L9ECAV7KIM	16	0 (0.00%)	45	31	97	37	0.30	0.00
GET	/product/LS4PSXNDNM	17	0 (0.00%)	57	31	272	38	0.20	0.00
GET	/product/OLJCESFC7Z	16	0 (0.00%)	39	30	102	34	0.20	0.00
POST	/setCurrency	23	0 (0.00%)	69	49	122	60	0.20	0.00
Aggregated		245	0 (0.00%)	58	27	306	48	2.60	0.00

Figure 2.10: Generated load

To verify that this load was successfully received by the frontend load balancer, we checked the frontend deployment in the Google Cloud Console and looked for two indicators:

1. Resource usage:

Both CPU and memory usage spiked, indicating that the load was being processed.

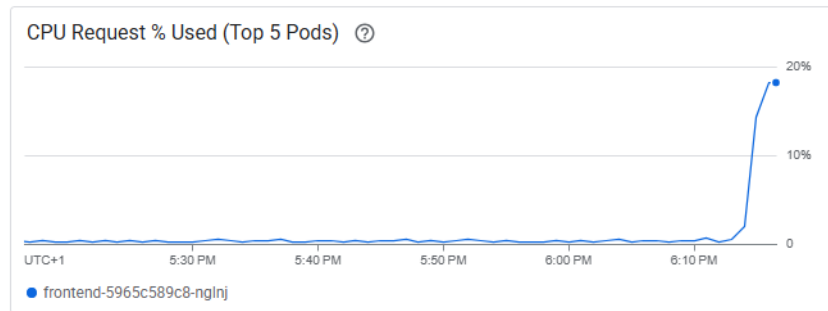


Figure 2.11: CPU Usage

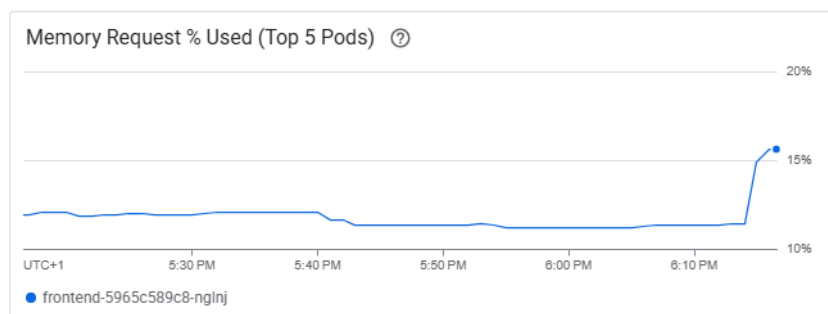
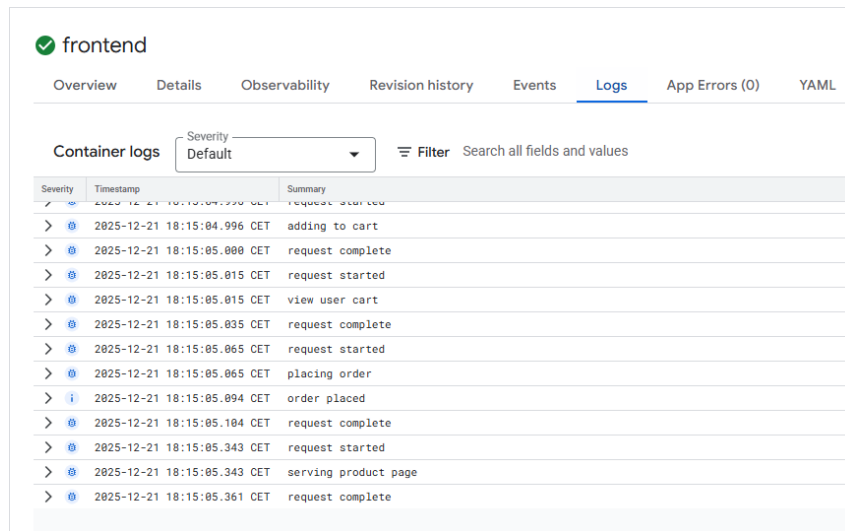


Figure 2.12: Memory Usage

2. Logs:

By inspecting the logs, we confirmed that the requests were indeed received and processed.



The screenshot shows the 'frontend' application logs. The interface includes tabs for Overview, Details, Observability, Revision history, Events, Logs (selected), App Errors (0), and YAML. Under 'Container logs', there is a severity filter set to 'Default' and a search bar. The log entries are as follows:

Severity	Timestamp	Summary
> @	2025-12-21 18:15:04.996 CET	request started
> @	2025-12-21 18:15:04.996 CET	adding to cart
> @	2025-12-21 18:15:05.000 CET	request complete
> @	2025-12-21 18:15:05.015 CET	request started
> @	2025-12-21 18:15:05.015 CET	view user cart
> @	2025-12-21 18:15:05.035 CET	request complete
> @	2025-12-21 18:15:05.065 CET	request started
> @	2025-12-21 18:15:05.065 CET	placing order
> i	2025-12-21 18:15:05.094 CET	order placed
> @	2025-12-21 18:15:05.104 CET	request complete
> @	2025-12-21 18:15:05.343 CET	request started
> @	2025-12-21 18:15:05.343 CET	serving product page
> @	2025-12-21 18:15:05.361 CET	request complete

Figure 2.13: Frontend logs

Note: The script used to reproduce this step can be found in our repository [at:](#)

[base_steps/3_deploying_the_load_generator_on_a_local_machine/scripts/deploying_the_load_generator_on_a_local_machine.sh](#)

2.4 Deploying the Load Generator Automatically in Google Cloud

Before starting, we needed to create a service account that Terraform could use to provision and configure resources. We decided to use Terraform only, without Ansible, since our configuration needs were minimal and Terraforms [metadata startup script](#) was sufficient for this task. Using Ansible in this case would have been unnecessary overhead.

To create the service account, we followed the tutorial provided in the lab [1] and ran the following commands:

```
gcloud iam service-accounts create shopapp-terraform-account
gcloud projects add-iam-policy-binding $PROJECT_NAME \
  --member serviceAccount:shopapp-terraform-account@\
```

```
$PROJECT_NAME.iam.gserviceaccount.com --role roles/editor
```

Next, we created a terraform directory and downloaded the service account key using:

```
gcloud iam service-accounts keys create ./shopapp-terraform-account\
.json --iam-account shopapp-terraform-account@\
$PROJECT_NAME.iam.gserviceaccount.com
```

Which gave a json file containing the credentials that are going to be used by Terraform.

Inside this directory, we placed two Terraform files. The first one, **variables.tf**, contains variables such as the project ID, zone/region, instance name, and the frontend IP address. The second file, main.tf, is responsible for provisioning and configuring the virtual machine:

```
resource "google_compute_instance" "vm_instance" {  
  name = var.instance-name  
  
  machine_type = "e2-medium"  
  
  boot_disk {  
    initialize_params {  
      image = "ubuntu-2204-lts"  
    }  
  }  
  
  network_interface {  
    network = "default"  
    access_config {  
    }  
  }  
  
  metadata_startup_script = <<-EOT  
  #!/bin/bash  
  apt-get update  
  
  # Install docker  
  apt-get install -y docker.io git  
  systemctl start docker  
  systemctl enable docker  
  
  # Clone the project and move to the loadgenerator folder  
  cd /tmp  
  git clone --depth 1 --branch v0 https://github.com/GoogleCloudPlatform/microservices-demo.git  
  cd microservices-demo/src/loadgenerator  
  
  # Modify the Dockerfile to remove the platform argument for compatibility  
  sed -i 's/--platform=$BUILDPLATFORM //' Dockerfile  
  
  # Build and run the container  
  docker build -t loadgenerator .  
  docker run --rm -e FRONTEND_ADDR=${var.frontend_ip} loadgenerator  
  EOT  
}
```

Figure 2.14: main.tf

As shown in the initialization (startup) script, it installs the required packages such as Docker and Git, then clones the project repository, and finally builds and runs the loadgenerator container.

```
sed -i 's/--platform=$BUILDPLATFORM //' Dockerfile
```

This line removes `--platform=$BUILDPLATFORM` from the Dockerfile. This was necessary because it was the source of a build error we encountered. After further investigation, we found that `$BUILDPLATFORM` is a Buildx

variable. When BuildKit is disabled, this variable expands to an empty string, which causes a Dockerfile parsing error. Removing it resolved the issue.

```
FROM --platform=$BUILDPLATFORM python:3.12.8-alpine@sha256:  
  
FROM base AS builder  
  
RUN apk update \  
    && apk add --no-cache wget g++ linux-headers \  
    && rm -rf /var/cache/apk/*  
  
COPY requirements.txt .
```

Figure 2.15: Dockerfile problematic part

We then ran the deployment using:

```
terraform apply
```

As in the previous step, we verified that the load was both generated and received by the frontend. By checking the frontend logs, we confirmed that the requests were correctly received and processed.

✔ frontend

Overview Details Observability Revision history Events **Logs** App Errors (0) YAML

Container logs Severity: Default Filter Search all fields and values

Severity	Timestamp	Summary
> ⓘ	2025-12-21 22:15:53.487 CET	serving product page
> ⓘ	2025-12-21 22:15:53.505 CET	request complete
> ⓘ	2025-12-21 22:15:53.868 CET	request started
> ⓘ	2025-12-21 22:15:53.868 CET	serving product page
> ⓘ	2025-12-21 22:15:53.884 CET	request complete
> ⓘ	2025-12-21 22:15:53.886 CET	request started
> ⓘ	2025-12-21 22:15:53.886 CET	adding to cart
> ⓘ	2025-12-21 22:15:53.890 CET	request complete
> ⓘ	2025-12-21 22:15:53.892 CET	request started
> ⓘ	2025-12-21 22:15:53.893 CET	view user cart
> ⓘ	2025-12-21 22:15:53.910 CET	request complete
> ⓘ	2025-12-21 22:15:53.928 CET	request started
> ⓘ	2025-12-21 22:15:53.928 CET	request complete
> ⓘ	2025-12-21 22:15:54.285 CET	request started
> ⓘ	2025-12-21 22:15:54.285 CET	serving product page
> ⓘ	2025-12-21 22:15:54.305 CET	request complete
> ⓘ	2025-12-21 22:15:55.980 CET	request started
> ⓘ	2025-12-21 22:15:55.980 CET	serving product page
> ⓘ	2025-12-21 22:15:55.998 CET	request complete
> ⓘ	2025-12-21 22:15:56.424 CET	request started
> ⓘ	2025-12-21 22:15:56.424 CET	serving product page

Figure 2.16: Frontend logs 2

We also connected to the virtual machine via SSH and ran `docker ps`, which confirmed that the loadgenerator container was running.

```
soufiane@loadgenerator-vm:~$ sudo docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
8f09c290a67a	loadgenerator	"/bin/sh -c 'locust ...'"	About a minute ago	Up About a minute		elastic_goldstine

Figure 2.17: Container running inside the VM

Note: The script used to reproduce this step can be found in our repository at:

[base_steps/4_deploying_automatically_the_load_generator_in_google_cloud/scripts/deploying_automatically_the_load_generator_in_google_cloud.sh](#)

Chapter 3

Advanced steps

3.1 Monitoring the application and the infrastructure

3.1.1 Design Choices

For the design, it is clear from the task requirements that we need exporters at both the node and pod levels in our cluster. Prometheus and Grafana, also deployed within the same cluster, will collect and visualize these metrics.

3.1.2 Technical Choices

For this part, we researched how to deploy a monitoring stack in our GKE cluster. We found two possible approaches:

1. **Manual Deployment:**

This involves installing exporters for node and pod level metrics, then deploying Prometheus to collect the metrics, and finally deploying Grafana to visualize them on dashboards.

2. **Using Helm:**

Helm is a package manager for Kubernetes, similar to apt or npm. It simplifies application deployment by bundling apps into reusable packages called Charts. We found a Helm Chart called

kube-prometheus-stack that performs all the tasks described in the first approach automatically. We would only need to install Helm, deploy the Chart on our cluster..

We chose the second option for two main reasons: first, it is simpler and saves time, and second, we had already implemented the manual approach in a previous cloud project ("*Systèmes distribués pour le traitement des données*") and wanted to try Helm since we hadn't used it before.

3.1.3 Implementation

The first step was to install Helm on our machine to use it from the terminal, using the following commands:

```
curl -fsSL -o get_helm.sh https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-4
chmod 700 get_helm.sh
./get_helm.sh
```

According to Helm's quickstart guide [2], we first need to add the repository containing the Chart before installing it. In Helm, installing a Chart means deploying it to the cluster.

To access the desired Chart, we added the Prometheus repository to Helm following the README instructions on their GitHub repo [3] and verified it:

```
soufiane@DESKTOP-RNOR9SM:~$ helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
"prometheus-community" has been added to your repositories
soufiane@DESKTOP-RNOR9SM:~$ helm search repo prometheus-community
```

NAME	CHART VERSION	APP VERSION	DESCRIPTION
prometheus-community/alertmanager	1.30.0	v0.30.0	The Alertmanager handles alerts sent by client ...
prometheus-community/alertmanager-snmp-notifier	2.1.0	v2.1.0	The SNMP Notifier handles alerts coming from Pr...
prometheus-community/jiralert	1.8.1	v1.3.0	A Helm chart for Kubernetes to install jiralert
prometheus-community/kube-prometheus-stack	80.6.0	v0.87.1	kube-prometheus-stack collects Kuber...

Figure 3.1: Prometheus Charts

While reading the Prometheus community GitHub repo, we discovered a known issue when using GKE private clusters (our case). In private

clusters, the control plane is managed by Google in a different project, while the nodes are in our project. Because of firewall restrictions, the control plane cannot call the Admission Webhooks set up by Prometheus. These webhooks validate resource creation or modification requests.

The suggested solution is to disable the webhooks by setting `prometheusOperator.admissionWebhooks.enabled=false` during installation. Fortunately, we identified this before deploying the Chart.

To deploy the stack, we also created a separate namespace for monitoring:

```
helm install kube-prometheus-stack prometheus-community\
/kube-prometheus-stack --namespace monitoring \
--create-namespace \
--set prometheusOperator.admissionWebhooks.enabled=false
```

```
soufiane@DESKTOP-RNOR9SM:~$ helm install kube-prometheus-stack prometheus-community/kube-prometheus-stack --namespace monitoring --create-namespace --set pr
ometheusOperator.admissionWebhooks.enabled=false
NAME: kube-prometheus-stack
LAST DEPLOYED: Tue Dec 23 14:21:52 2025
NAMESPACE: monitoring
STATUS: deployed
REVISION: 1
DESCRIPTION: Install complete
NOTES:
kube-prometheus-stack has been installed. Check its status by running:
  kubectl --namespace monitoring get pods -l "release=kube-prometheus-stack"
Get Grafana 'admin' user password by running:
  kubectl --namespace monitoring get secrets kube-prometheus-stack-grafana -o jsonpath="{.data.admin-password}" | base64 -d ; echo
Access Grafana local instance:
  export POD_NAME=$(kubectl --namespace monitoring get pod -l "app.kubernetes.io/name=grafana,app.kubernetes.io/instance=kube-prometheus-stack" -oname)
  kubectl --namespace monitoring port-forward $POD_NAME 3000
Get your grafana admin user password by running:
  kubectl get secret --namespace monitoring -l app.kubernetes.io/component=admin-secret -o jsonpath="{.items[0].data.admin-password}" | base64 --decode ; ec
ho
Visit https://github.com/prometheus-operator/kube-prometheus for instructions on how to create & configure Alertmanager and Prometheus instances using the O
perator.
```

Figure 3.2: kube-prometheus-stack successfully installed

```
soufiane@DESKTOP-RNOR9SM:~$ kubectl -n monitoring get pods
```

NAME	READY	STATUS	RESTARTS	AGE
kube-prometheus-stack-grafana-7b9c96556d-lccg5	3/3	Running	0	17m
kube-prometheus-stack-kube-state-metrics-7846957b5b-9k4mt	1/1	Running	0	17m
kube-prometheus-stack-operator-5d85bbb4f4-cqgqz	0/1	ContainerCreating	0	17m
kube-prometheus-stack-prometheus-node-exporter-4jtm7	1/1	Running	0	17m
kube-prometheus-stack-prometheus-node-exporter-88j1h	1/1	Running	0	17m
kube-prometheus-stack-prometheus-node-exporter-v8v2v	1/1	Running	0	17m

Figure 3.3: kube-prometheus-stack pods

At this point, all pods were running except for the Prometheus operator.

Upon describing the pods, we found the issue:

```
MountVolume.SetUp failed for volume "tls-secret": secret  
"kube-prometheus-stack-admission" not found
```

This occurred due to disabling the Admission Webhook. After researching, we found a GitHub issue [4] suggesting that adding `prometheusOperator.tls.enabled=false` resolves the problem. We reinstalled the Chart with this option:

```
helm install kube-prometheus-stack prometheus-community\  
/kube-prometheus-stack \  
--namespace monitoring --create-namespace \  
--set prometheusOperator.admissionWebhooks.enabled=false \  
--set prometheusOperator.tls.enabled=false
```

To confirm that Prometheus was collecting metrics, we accessed the Prometheus UI. Since all services deployed by kube-prometheus-stack are of type ClusterIP, we used port forwarding:

```
kubectl -n monitoring port-forward svc\  
/kube-prometheus-stack-prometheus 9090:9090
```

Accessing localhost:9090 opened the Prometheus UI. Initially, it appeared empty, but after entering PromQL queries or searching metrics, we confirmed node and pod metrics were available. Examples include `node_cpu_seconds_total` and `container_cpu_usage_seconds_total`:

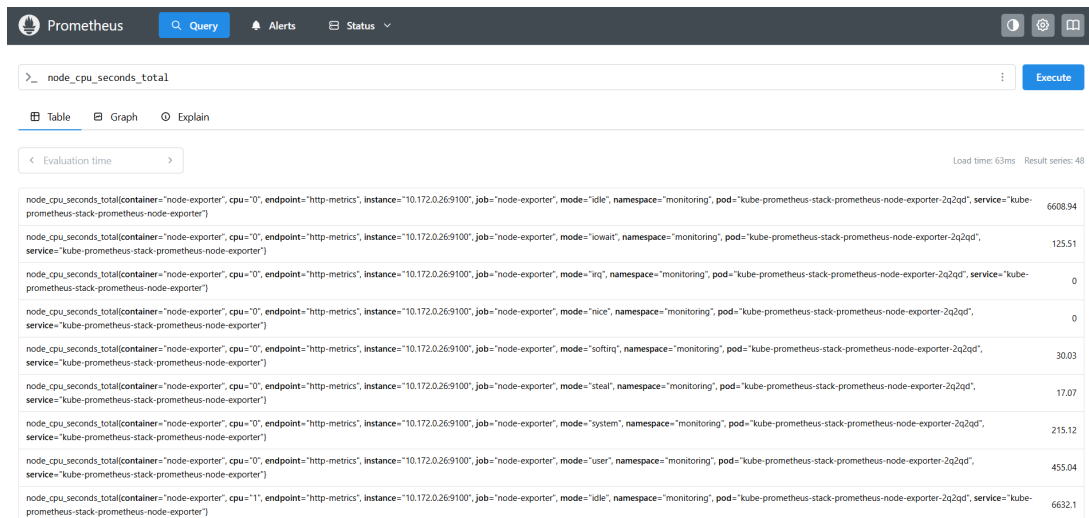


Figure 3.4: Node cpu seconds total

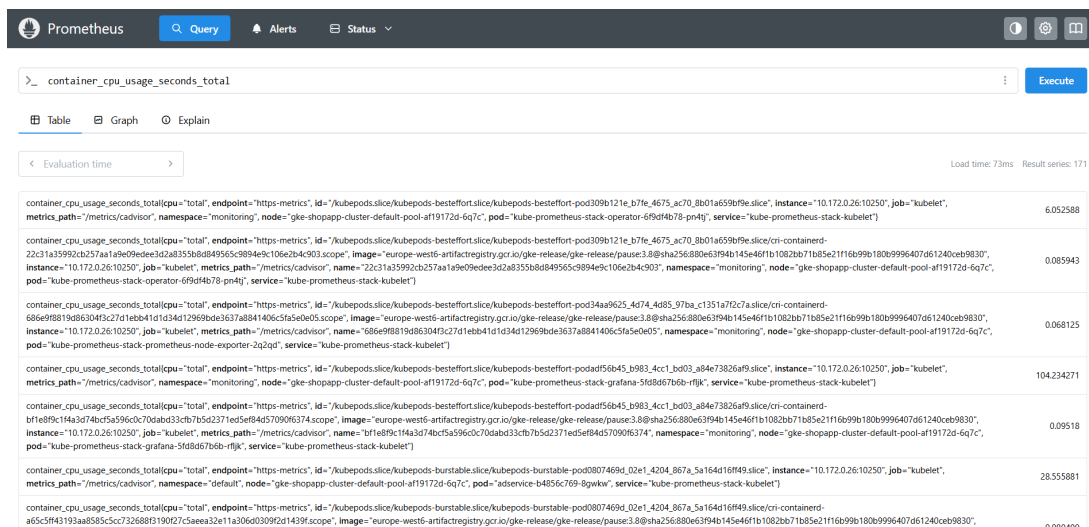


Figure 3.5: Container cpu usage seconds total

Finally, we used **Grafana** to visualize metrics. We retrieved the admin password:

```
kubectl --namespace monitoring get secrets \
kube-prometheus-stack-grafana \
-o jsonpath="{.data.admin-password}" | base64 -d ; echo
```

Port forwarding allowed access to the **Grafana** UI, where we used **pre-built** dashboards for visualizing node and pod resources. Two examples are **Node Exporter / Nodes** and **Kubernetes / Compute Resources / Pod** :

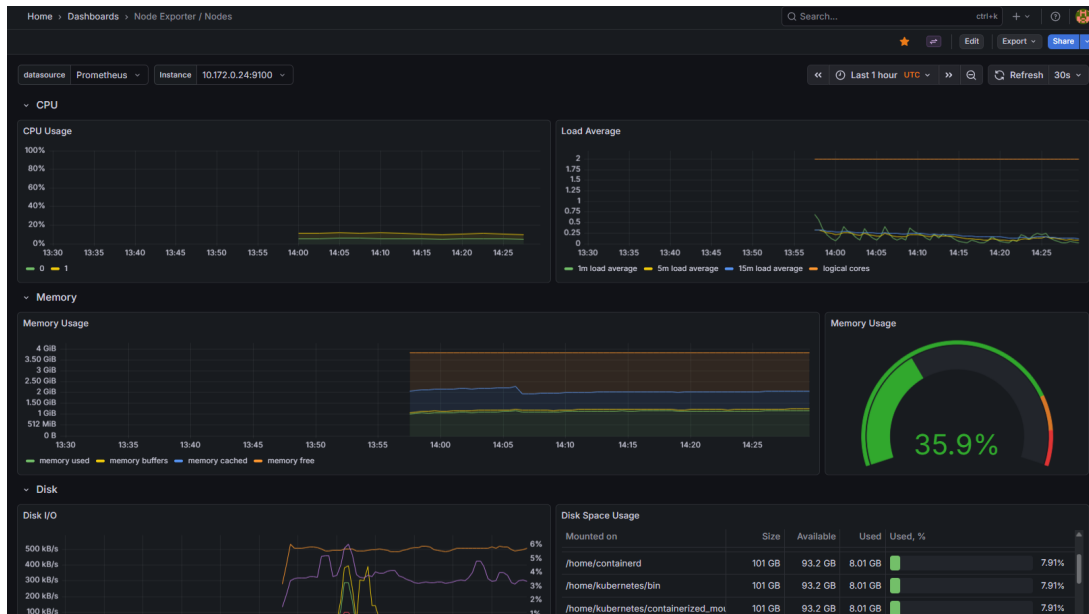


Figure 3.6: Nodes Resources Dashboard

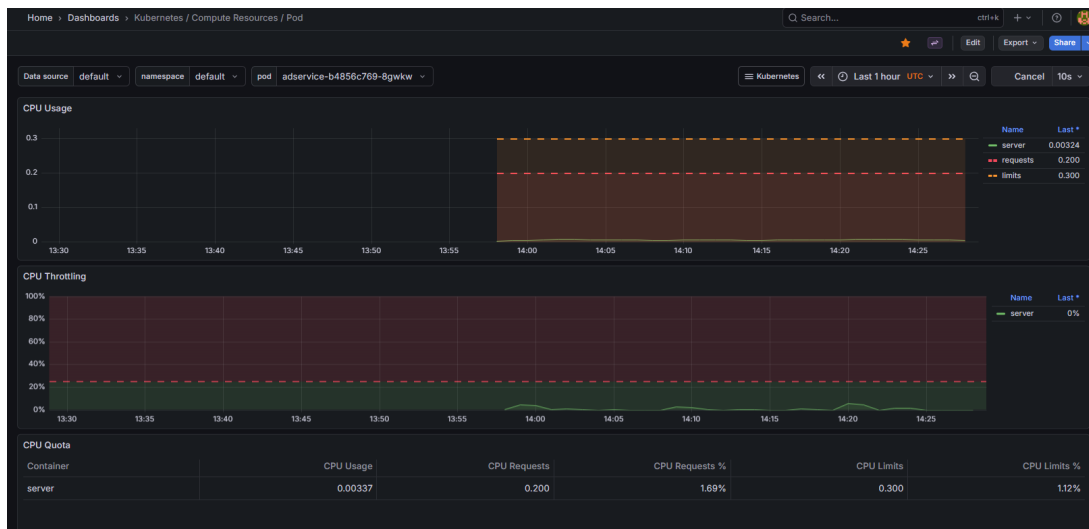


Figure 3.7: Pods Resources Dashboard

Note: The script used to reproduce this step can be found in our repository at:

`advanced_steps/1_monitoring_the_application_and_the_infrastructure/scripts/monitoring_the_application_and_the_infrastructure.sh`

3.2 Performance evaluation

3.2.1 Environment Preparation

Before running the evaluation, we needed to set up the environment. Specifically, we adapted the load generator from previous steps to be fully parameterized. In addition to specifying the target, we also needed to define the number of `users`, the `spawn rate`, the `run time`, and the `path to the CSV files`. We established a naming convention for the CSV files, using a prefix with the format `locust_numberofusers*.csv` (e.g., `locust_200_stats.csv` - `locust_200_failures.csv`).

We also found that a single VM was insufficient. Testing with one `e2-medium` VM failed for more than 200 users, causing the load generation to crash. To handle larger loads, we needed multiple VMs. Fortunately, Locust supports distributed load generation. We run one machine as the `master`, while other machines act as `workers`. The workers connect to the master using its IP address, and Locust automatically coordinates the distribution of users between the master and workers. Once the test completes, the workers send their results back to the master, which aggregates them into CSV files.

To implement this, we reused our previous Terraform files but adapted them for this scenario. We provisioned three virtual machines: a master and two workers. All machines set up their environment in the startup script by installing Docker and building the load generator Docker image. When running an experiment, we launch containers on both the master and workers VMs. The master container receives all the experiment variables as

environment variables, while the workers only need the masters IP address.

```
metadata_startup_script = <<-EOT
#!/bin/bash
apt-get update

# Install docker
apt-get install -y docker.io git
systemctl start docker
systemctl enable docker

# Clone project
cd /tmp
git clone --depth 1 --branch v0 https://github.com/GoogleCloudPlatform/microservices-demo.git
cd microservices-demo/src/loadgenerator

# Modify the Dockerfile to remove the platform argument for compatibility
sed -i 's/--platform=\$BUILDPLATFORM //' Dockerfile

# Modify ENTRYPOINT to use environment variables
sed -i '/^ENTRYPOINT /d' Dockerfile

cat <<'EOF' >> Dockerfile
ENTRYPOINT locust --master --host="http://\${FRONTEND_ADDR}" --headless -u "\${USERS:-10}" -r \\\
"\${RATE:-1}" -t "\${RUN_TIME:-5m}" --csv /tmp/results/locust_\${USERS:-10}
EOF

mkdir -p /tmp/results

# Build and run container
docker build -t loadgenerator .
EOT
```

Figure 3.8: Master's startup script

```
metadata_startup_script = <<-EOT
#!/bin/bash
apt-get update

# Install docker
apt-get install -y docker.io git
systemctl start docker
systemctl enable docker

# Clone project
cd /tmp
git clone --depth 1 --branch v0 https://github.com/GoogleCloudPlatform/microservices-demo.git
cd microservices-demo/src/loadgenerator

# Modify the Dockerfile to remove the platform argument for compatibility
sed -i 's/--platform=$BUILDPLATFORM //' Dockerfile

# Modify ENTRYPOINT to use environment variables
sed -i '/^ENTRYPOINT /d' Dockerfile

cat <<'EOF' >> Dockerfile
ENTRYPOINT locust --worker --master-host="${MASTER_IP_ADDRESS}"
EOF

mkdir -p /tmp/results

# Build and run container
docker build -t loadgenerator .
EOT
```

Figure 3.9: Worker’s startup script

To streamline the evaluation, we developed two automation scripts, a **driver** (`performance_evaluation.h`) and an **experiment runner** (`run_evaluation.sh`). The driver defines the experiments to be executed, creates the Terraform resources, and waits for the startup scripts to finish on all VMs. It then executes each experiment using the runner script. The runner takes the master IP, number of workers, number of users, spawn rate, and run time as input. It starts the containers on the worker machines, which wait for the master to be ready, then launches the container on the master. Once the experiment completes, the runner copies the CSV results back to the local machine. After all experiments finish, the driver script destroys the resources.

3.2.2 Evaluation Plan and Experiment Design

With the environment prepared, we defined our evaluation strategy.

We plan to execute 17 experiments with varying numbers of users. Each experiment runs for 5 minutes, and the spawn rate is set to users / 10, ensuring that all users are active within 10 seconds. The experiments are as follows:

Experiment	Users	Spawn Rate	Run Time
E1	10	1	5m
E2	25	2	5m
E3	50	5	5m
E4	100	10	5m
E5	200	20	5m
E6	300	30	5m
E7	400	40	5m
E8	500	50	5m
E9	600	60	5m
E10	700	70	5m
E11	800	80	5m
E12	900	90	5m
E13	1000	100	5m
E14	1500	150	5m
E15	2000	200	5m
E16	5000	500	5m
E17	10000	1000	5m

Table 3.1: Experiments

After executing all experiments, we will generate comparison plots from the resulting CSV files. Additionally, we will monitor resource consumption through Grafana dashboards. The start and end times of each experiment are recorded to facilitate fetching metrics and visualizing results for each specific experiment.

3.2.3 Results Analysis

3.2.3.1 CPU-Level Performance Analysis

This section analyzes the CPU behavior of the system under increasing workload intensity, using data collected from both Grafana monitoring dashboards. The objective is to understand how CPU utilization evolves at the service, and node levels as request rates increase, and to identify potential bottlenecks or saturation points in the system.

CPU Utilization Across Services

The figure below presents a heatmap summarizing the average CPU usage (expressed as a percentage of requested CPU) for each microservice across all experiments.



Figure 3.10: CPU Heatmap

The heatmap reveals several important trends:

- **Frontend**, **recommendationservice**, and **currencyservice** are consistently the most CPU-intensive components. Their CPU usage

increases rapidly with load and peaks during intermediate experiments (E5E9), where concurrent users range from 200 to 600.

- The **recommendationservice** shows the highest overall CPU demand, reaching values exceeding 300% of requested CPU, indicating that it is the most computationally intensive service and a potential performance bottleneck.
- The **frontend** service exhibits sustained high utilization across nearly all experiments, confirming its central role in request handling and aggregation.
- In contrast, services such as **emailservice**, **shippingservice**, **paymentservice**, and **redis** maintain low and relatively stable CPU usage throughout all experiments, indicating that they are not critical bottlenecks under the tested workloads.
- At higher loads (experiments E12E17), a noticeable drop in CPU utilization is observed across most services. This reflects system saturation, where request throughput no longer increases proportionally with load due to queuing, throttling, or resource contention.

This behavior demonstrates a classical saturation curve: rapid utilization growth under increasing load, followed by stabilization or decline once the system reaches its processing limits.

CPU Usage Evolution Across Experiments

The figure below illustrates the evolution of CPU usage for each service across all experiments.

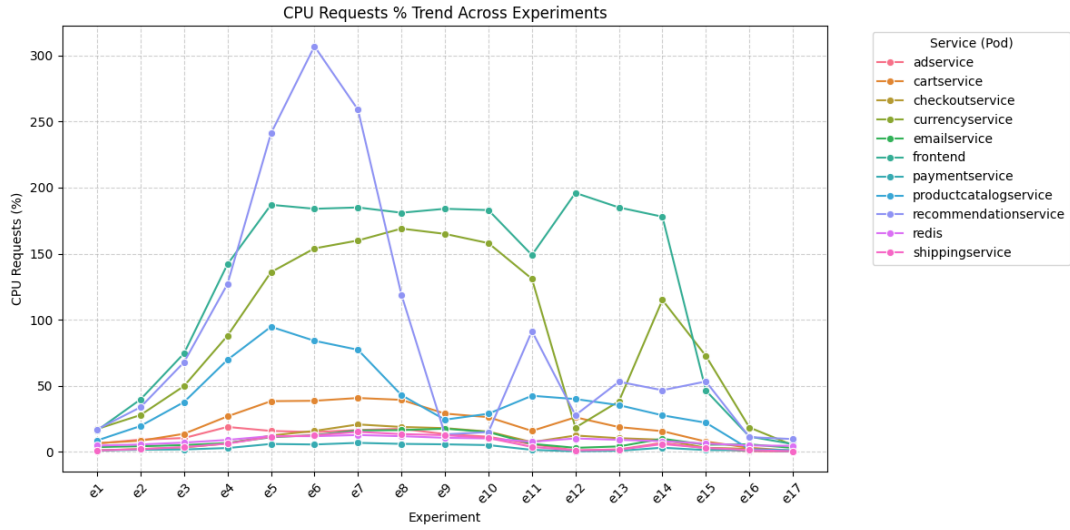


Figure 3.11: CPU requests per workload

Several key patterns emerge:

- **From E1 to E6**, CPU usage increases almost linearly for most services, indicating efficient horizontal scaling and sufficient available resources.
- **Between E6 and E10**, several services (notably frontend, recommendationservice, and productcatalogservice) begin to plateau, signaling that they are approaching their CPU limits.
- **After E10**, CPU usage for many services declines despite increasing load. This counterintuitive behavior is a strong indicator of saturation: incoming requests are either queued, delayed, or dropped before reaching certain microservices.
- The recommendation service shows the most pronounced spike and subsequent decline, reinforcing its role as a critical performance bottleneck.

This trend demonstrates that CPU becomes the dominant limiting resource before memory or network saturation, which aligns with expectations for compute-intensive microservices.

Node-Level CPU Utilization

Figures below show CPU utilization over time for each Kubernetes node.



Figure 3.12: CPU usage of node 1

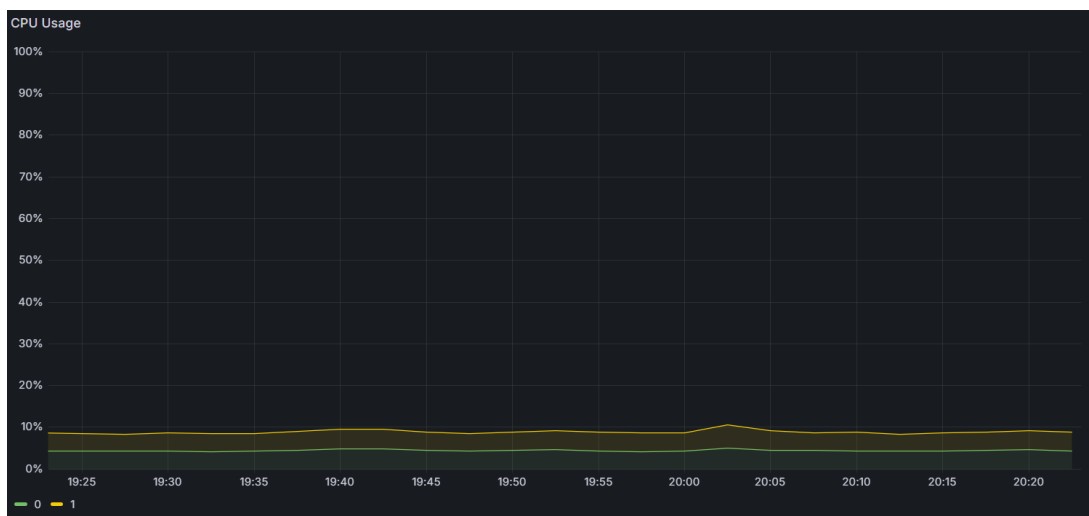


Figure 3.13: CPU usage of node 2

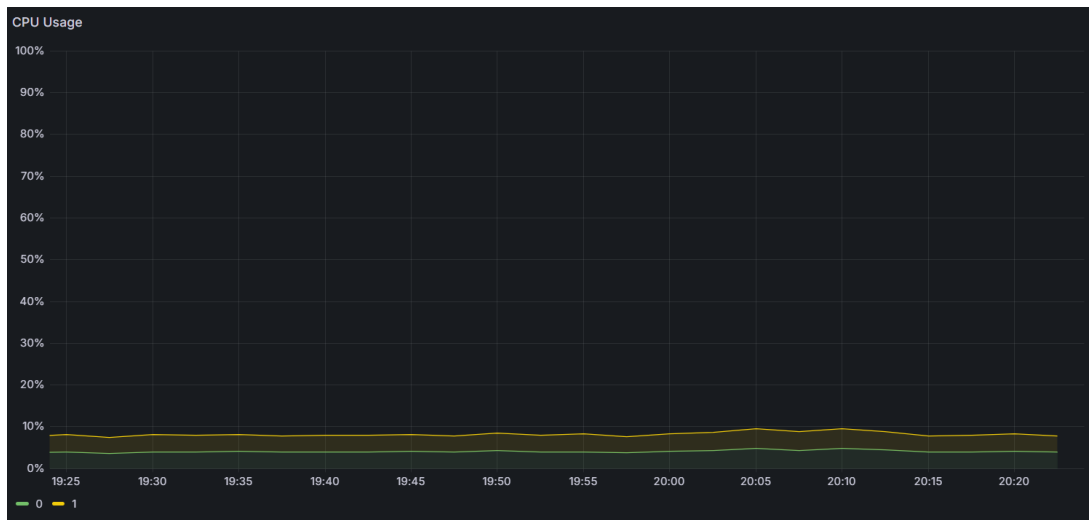


Figure 3.14: CPU usage of node 3

Across all nodes, CPU utilization remains relatively balanced, with no single node becoming a hotspot. The following observations can be made:

- CPU utilization gradually increases as load ramps up, peaking around 810
- The smooth and consistent curves indicate effective Kubernetes scheduling and load distribution across nodes.
- No sustained CPU saturation or instability is observed at the node level, confirming that resource pressure originates primarily at the pod level rather than from node exhaustion.

This suggests that the clusters node capacity is sufficient and that scaling limitations arise from application-level constraints rather than infrastructure shortages.

3.2.3.2 Memory-Level Analysis

This section analyzes the memory behavior of the system under increasing load, using metrics collected from Prometheus and visualized through Grafana dashboards. The objective is to assess memory

consumption trends at both the service and node levels, identify potential bottlenecks, and determine whether memory pressure contributes to performance degradation observed in high-load scenarios.

Memory Utilization Across Services

The figure below presents the evolution of memory usage (expressed as a percentage of requested memory) for each microservice across the different experimental workloads.

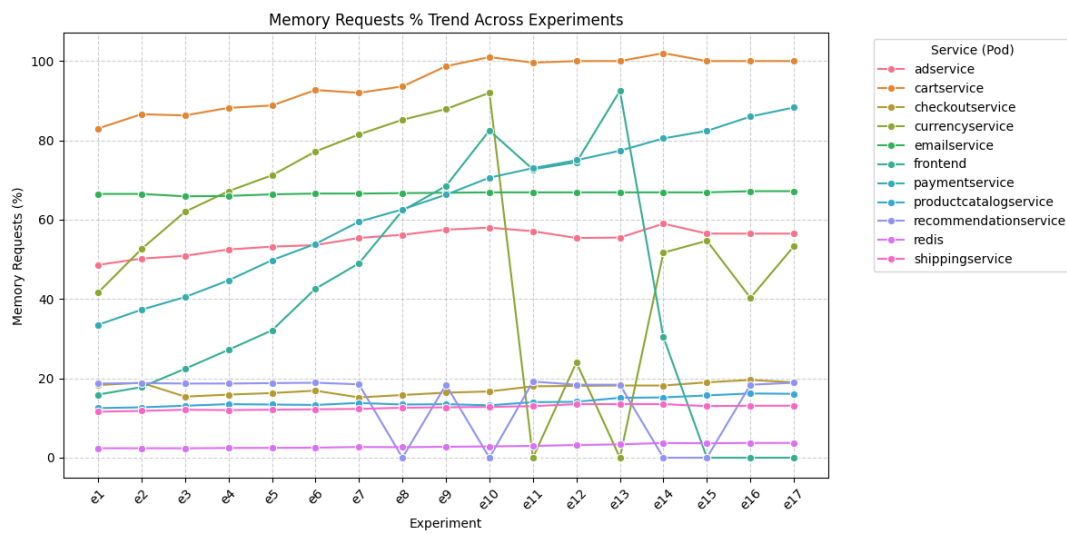


Figure 3.15: Memory requests per workload

Several important observations can be drawn:

- Most services exhibit stable and predictable memory consumption, even as the number of concurrent users increases significantly
- Core services such as frontend, checkoutservice, cartservice, and productcatalogservice show a gradual increase in memory usage up to a plateau, after which consumption stabilizes.
- Unlike CPU usage, memory usage does not exhibit sharp spikes or saturation effects, indicating that memory is not a limiting factor in system performance.

- The `recommendationservice` and `currencyservice` show moderate variability, with noticeable increases at mid-range load levels, but still remain well within safe operational bounds
- Memory usage for lighter services (e.g., `emailservice`, `redis`, `shippingservice`) remains consistently low and stable throughout all experiments.

Overall, memory consumption scales smoothly with load and does not exhibit signs of exhaustion or excessive pressure.

Node-Level Memory Utilization

Figures below present the memory usage evolution for each Kubernetes node over time.

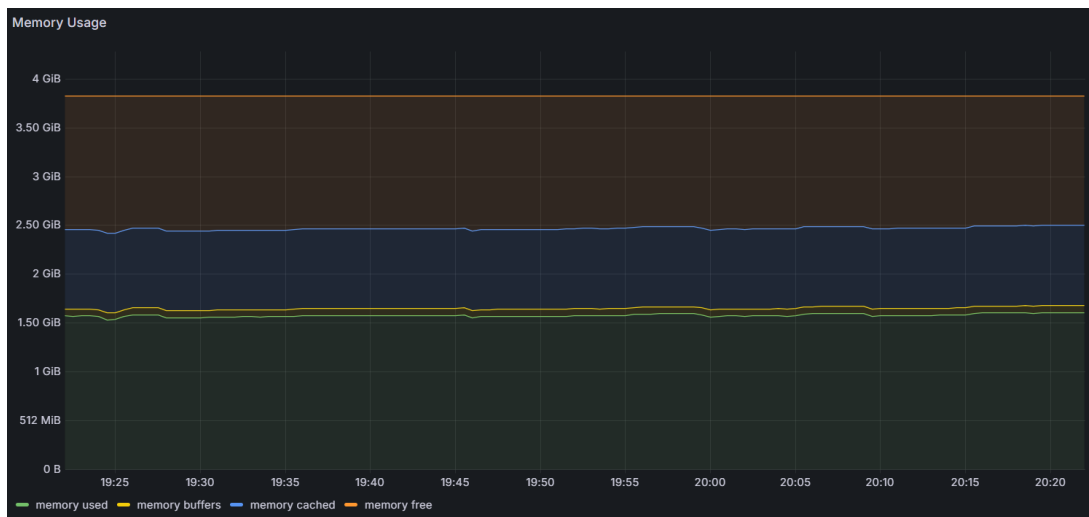


Figure 3.16: Memory usage of node 1

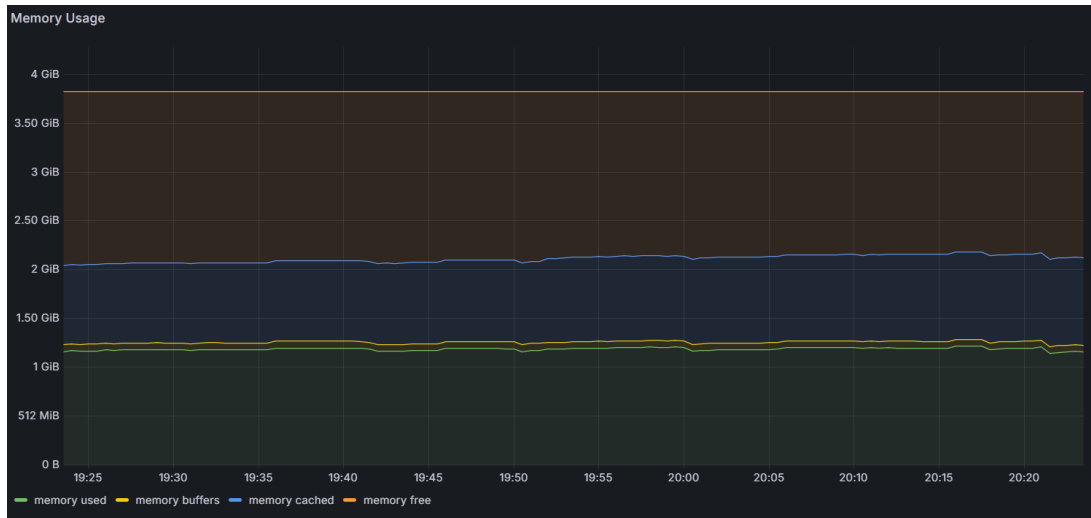


Figure 3.17: Memory usage of node 2

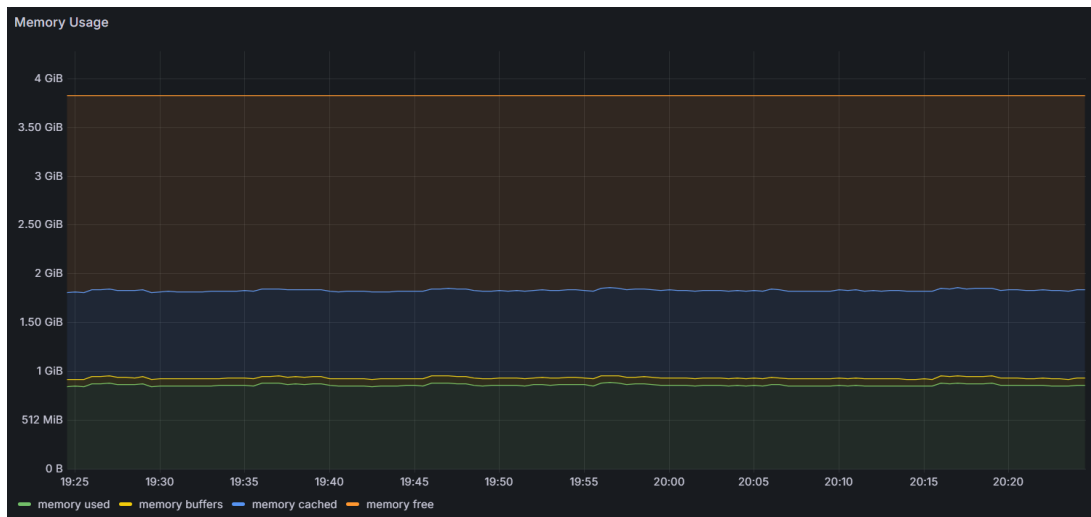


Figure 3.18: Memory usage of node 3

Across all nodes, memory usage remains remarkably stable throughout the experiment. The absence of memory saturation strongly suggests that **the system is not memory-bound**, even at the highest tested loads.

3.2.3.3 Network-Level Analysis

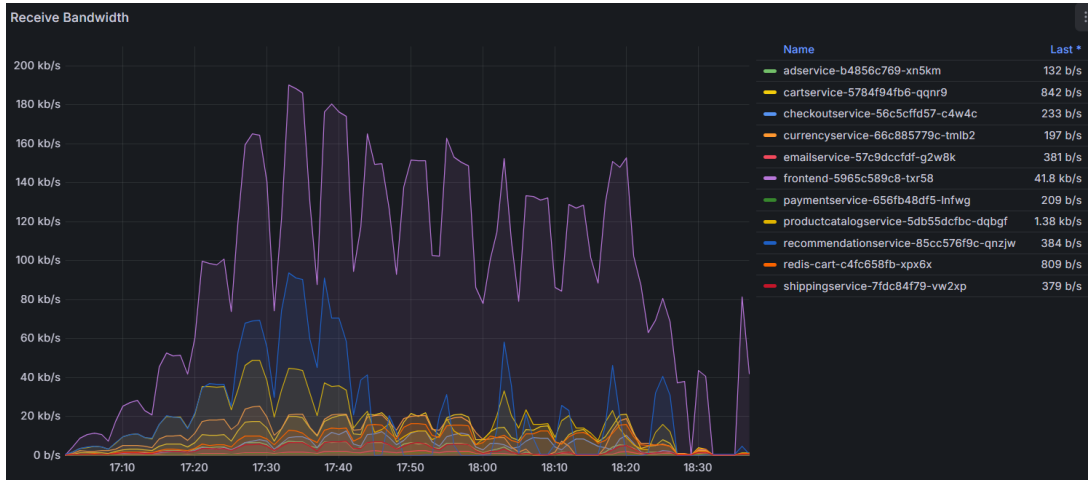


Figure 3.19: Receive bandwidth for each microservice

The results show that network traffic closely follows the request intensity and mirrors the behavior observed in the CPU-level analysis. The frontend and recommendationservice clearly dominate network usage, reaching peak inbound bandwidth values of approximately 150180 kB/s during the highest load phases. This is expected, as these components act as aggregation and routing points for user requests and internal service calls.

Other services such as productcatalogservice, cartservice, and checkoutservice exhibit moderate and stable network usage, reflecting their role in handling structured data exchanges triggered by user interactions. In contrast, lightweight services such as emailservice, shippingservice, and redis maintain consistently low bandwidth consumption throughout all experiments, indicating minimal network overhead.

Overall, the network behavior remains stable and proportional to request intensity, with no signs of congestion, packet loss, or saturation. The absence of abrupt drops or sustained plateaus suggests that the

underlying network infrastructure is not a bottleneck for system performance. This confirms that, under the tested workloads, network capacity is sufficient and that performance limitations observed elsewhere are more strongly correlated with CPU-bound components rather than network constraints.

3.2.3.4 Application-Level Performance Analysis

This section analyzes the end-to-end behavior of the application under increasing load, based on metrics collected by the Locust load generator. The analysis focuses on three key performance indicators: request failure rate, response latency, and system throughput. Together, these metrics provide a comprehensive view of how the application behaves under stress and where its scalability limits emerge.

Failure Rate Analysis

The figure below presents the evolution of the request failure rate as a function of the number of concurrent users.

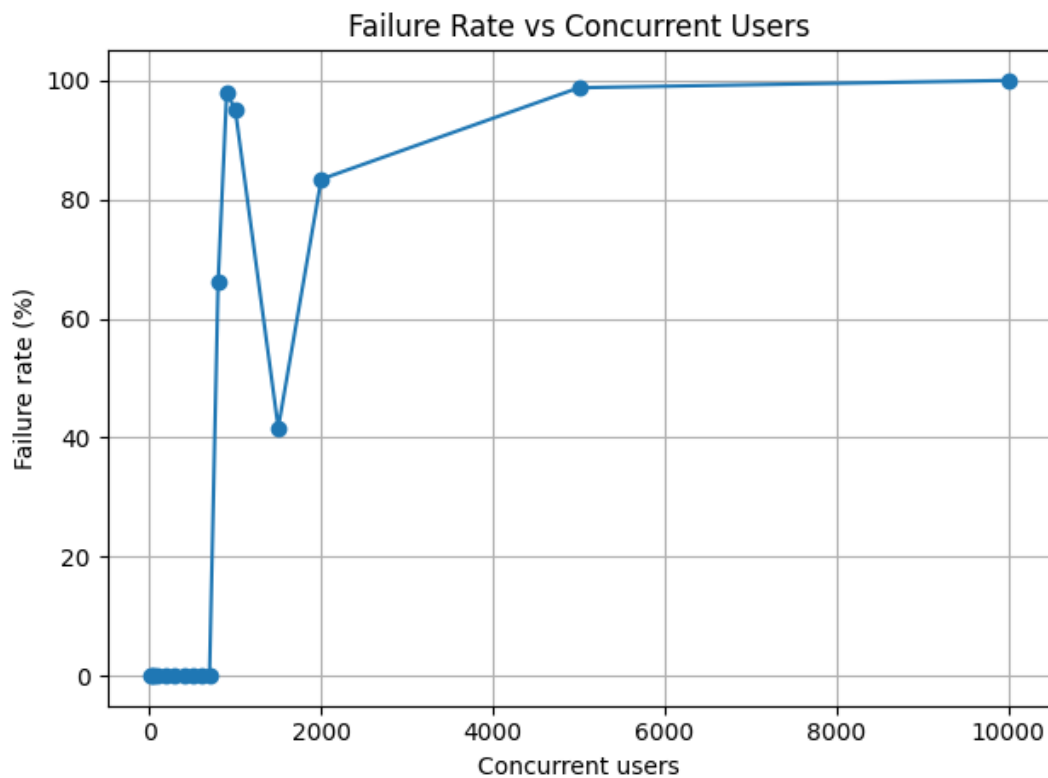


Figure 3.20: Failure rate vs users

At **low to moderate** load levels (up to approximately 500 concurrent users), the failure rate remains close to zero, indicating that the system is able to successfully process nearly all incoming requests. This confirms that, under light and moderate traffic, the application operates within its capacity limits and maintains reliable service behavior.

However, as concurrency increases beyond approximately 1,000 users, the failure rate **rises sharply**. At around 1,500 concurrent users, more than 40% of requests begin to fail, and beyond 2,000 users, the failure rate approaches nearly 100%, indicating **near-total saturation** of the system. This sharp transition reflects a classic overload scenario where incoming requests exceed the systems processing capacity.

The steep slope of the failure curve suggests that the bottleneck is not gradual degradation but a hard capacity limit, most likely caused by saturation of critical backend services (as observed earlier in the CPU analysis), connection exhaustion, or request queue overflows.

Latency Behavior Under Load

The figure below illustrates the evolution of request latency (p50, p95, and p99 percentiles) as a function of the number of concurrent users.

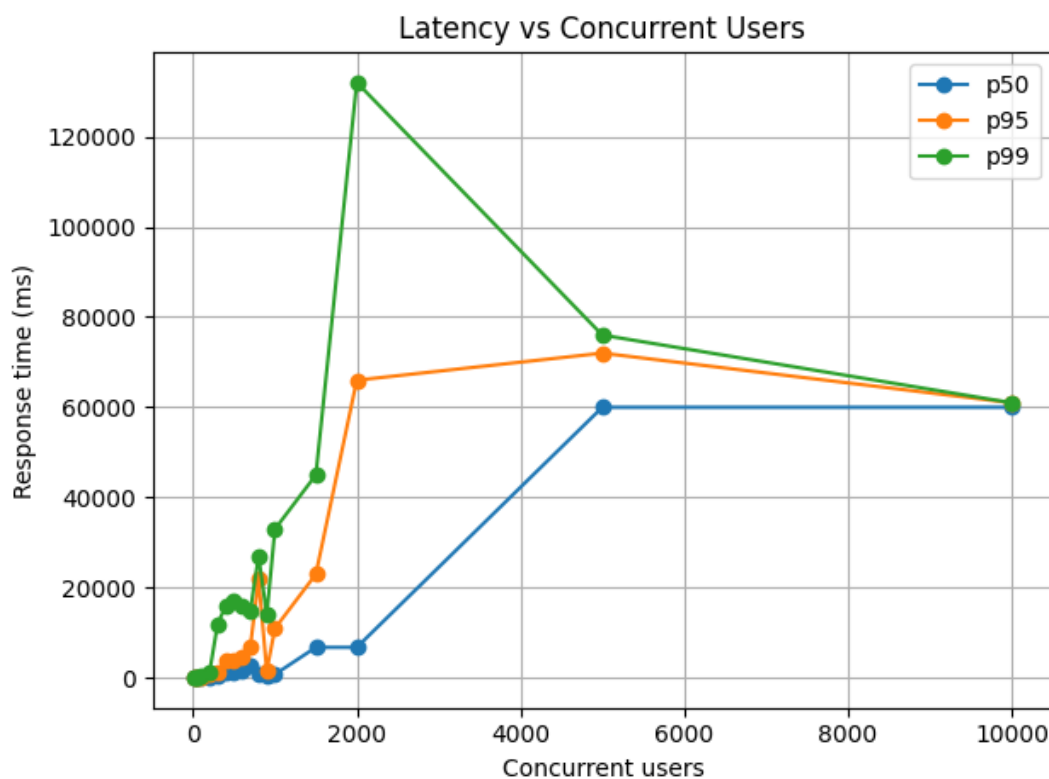


Figure 3.21: Latency vs users

Several important trends emerge:

- At low concurrency levels (below 500 users), response times remain low and stable, with median latency well below 1 second.

- As concurrency increases, latency begins to grow gradually, with the p95 and p99 percentiles rising faster than the median, indicating increasing variability in response times.
- Beyond approximately 1,000 concurrent users, latency increases sharply, with p99 latency exceeding 100 seconds in the most extreme cases. This behavior signals severe queuing delays and backend saturation.
- The divergence between p50 and p99 latencies at high load demonstrates that while some requests still complete relatively quickly, a growing fraction experience very long delays or timeouts.

This pattern is characteristic of systems operating beyond their sustainable throughput, where internal queues grow rapidly and response times degrade non-linearly.

Throughput Analysis

The figure below shows the evolution of achieved throughput (requests per second) as the number of concurrent users increases.

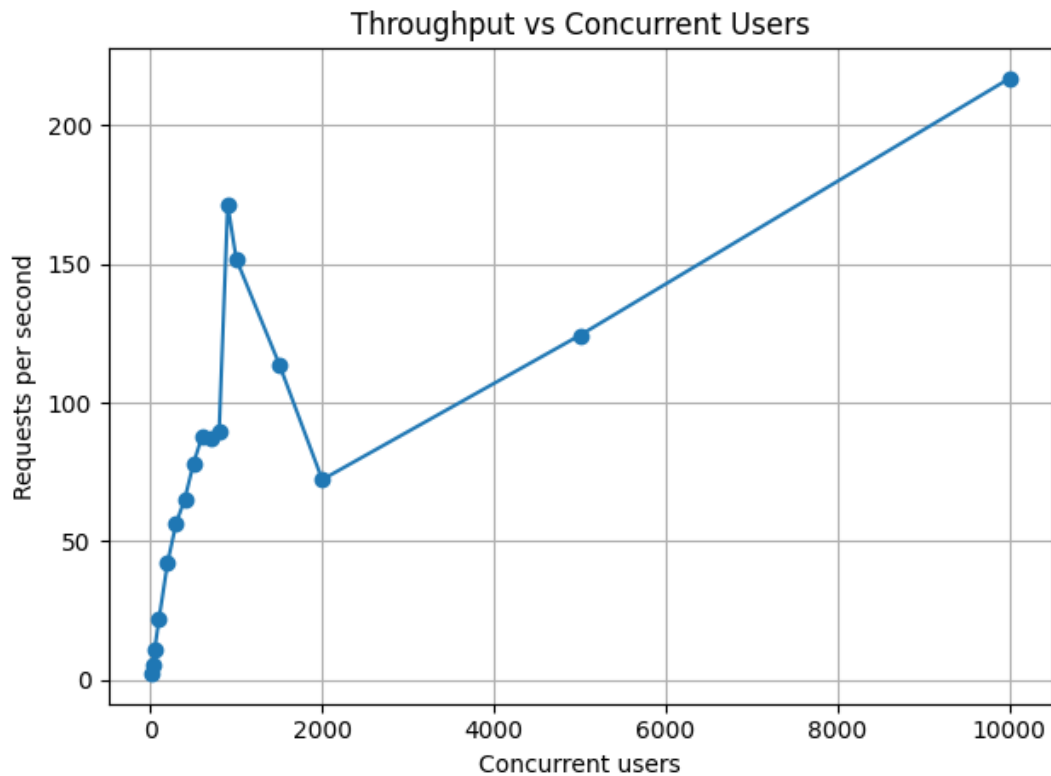


Figure 3.22: Throughput vs users

Key observations include:

- Throughput increases almost linearly up to approximately 8001000 concurrent users, indicating efficient scaling and good resource utilization in this range.
- Beyond this point, throughput growth slows down and eventually plateaus, even as additional users are added.
- In some regions, throughput slightly decreases despite increasing load, confirming that the system has reached its maximum processing capacity.

This saturation behavior aligns closely with the sharp rise in failure rate and latency observed in previous sections, confirming that the system has reached its operational limit.

3.2.3.5 Overall Interpretation and Summary

The experimental results show that the system behaves predictably and consistently under increasing load. At low to moderate concurrency levels, the application performs efficiently, with stable response times, low failure rates, and balanced resource utilization. CPU usage increases proportionally with load, while memory and network resources remain largely underutilized, indicating that they are not limiting factors.

As the number of concurrent users grows beyond roughly 8001000, the system reaches a saturation point. At this stage, CPU resources particularly in core services such as the frontend and recommendation components become the primary bottleneck. This leads to a sharp increase in response latency and request failures, while throughput plateaus or slightly decreases.

Overall, the results demonstrate that the application is CPU-bound rather than memory- or network-limited. The system scales predictably up to its capacity threshold, after which performance degrades due to computational constraints. These findings highlight clear opportunities for optimization through horizontal scaling, load balancing, or optimization of CPU-intensive services.

Note: The script used to reproduce this step can be found in our repository at:

`advanced_steps/2_performance_evaluation/scripts/performance_evaluation.sh`

3.3 Canary Releases

3.3.1 Setup and Microservice Preparation

To enable canary releases, we installed Istio and enabled automatic sidecar injection in the default namespace. This setup allows Istio to transparently manage and split traffic between different versions of a

microservice, which is essential for controlled rollouts and observability. The following commands were used:

```
curl -L https://istio.io/downloadIstio | ISTIO_VERSION=1.18
export PATH=$PATH:./istio-1.18.0/bin
istioctl install --set profile=demo -y
kubectl label namespace default istio-injection=enabled --overwrite
```

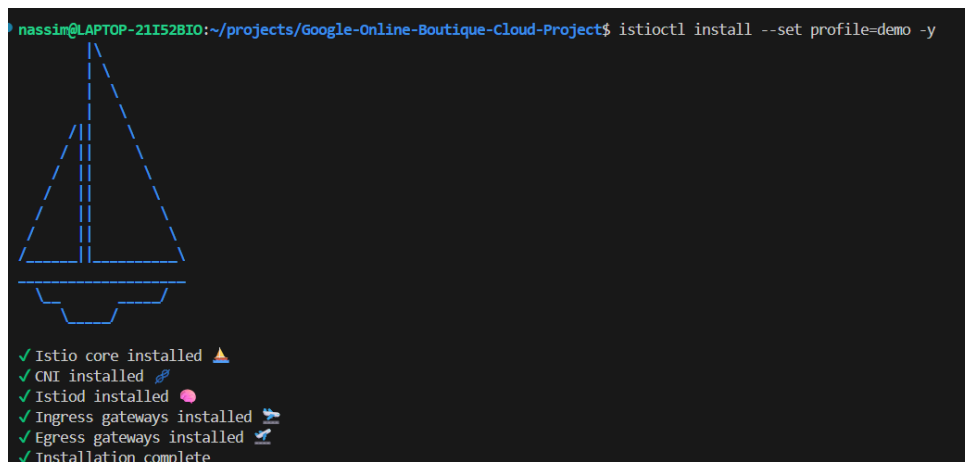


Figure 3.23: istio installed in the cluster

For the canary experiment, we chose the **frontend** microservice. This choice was motivated by the fact that it is stateless, making it easier to run multiple versions at the same time without worrying about user sessions or shared data. To create a visible difference, a string in the code was modified for the v2 version. The new image was built and pushed to the container registry:

```
docker build -t gcr.io/cloud-k8s-project-478408/frontend:v2 .
docker push gcr.io/cloud-k8s-project-478408/frontend:v2
```

```

nassing@LAPTOP-21152B10:~/projects/microservices-demo/src/emailservice$ docker build -t gcr.io/$PROJECT_ID/emailservice:v2 .
-> exporting layers
-> exporting manifest sha256:eb61702d939b070ada7422245477fca8cb05ab1f05957b7fc777f66d52f7431 0.0%
-> exporting config sha256:e54175bce09648a1552195d83df832d14e9178b7db65bd7fabe1099aef972820 0.0%
-> exporting attestation manifest sha256:8d24e5649e21b9c0de9be679ba3e3b094a568d96e8a1c71fa967aac251f2bc76 0.0%
-> exporting manifest list sha256:a1f2df470b8381293811605c4bf2581ab4d65d2f9237b43efa80fb82c2f7c196 0.0%
-> naming to gcr.io/cloud-k8s-project-478408/emailservice:v2 0.0%
-> unpacking to gcr.io/cloud-k8s-project-478408/emailservice:v2 0.0%
nassing@LAPTOP-21152B10:~/projects/microservices-demo/src/emailservice$ docker push gcr.io/cloud-k8s-project-478408/emailservice:v2
The push refers to repository [gcr.io/cloud-k8s-project-478408/emailservice]
785a7b384586: Pushed
22ba1c2e1d45: Layer already exists
2239f96b0057: Pushed
244d09e1ad68: Pushed
395e1cb4866: Pushed
2d35ebdb57d9: Layer already exists
f99639dbba01: Layer already exists
362fe14d6ada: Layer already exists
7026deb94eaf: Pushed
v2: digest: sha256:a1f2df470b8381293811605c4bf2581ab4d65d2f9237b43efa80fb82c2f7c196 size: 856

```

Figure 3.24: frontend v2 image build and push.

Deploying both v1 and v2 required more resources than the default cluster provided, so we increased the number of nodes to ensure both versions could run without resource constraints:

```

gcloud container clusters resize shopapp-cluster \
--node-pool=default-pool --num-nodes=4 \
--zone=europe-west6-a

```

```

nassing@LAPTOP-21152B10:~/projects/Google-Online-Boutique-Cloud-Project/microservices-demo/istio manifests$ gcloud container clusters resize shopapp-cluster --node-pool=default-pool --num-nodes=4 --zone=europe-west6-a
Pool [default-pool] for [shopapp-cluster] will be resized to 4 node(s) in each zone it spans.
Do you want to continue (y/n)? y
Resizing shopapp-cluster...done.
Updated [https://container.googleapis.com/v1/projects/cloud-k8s-project-478408/zones/europe-west6-a/clusters/shopapp-cluster].

```

Figure 3.25: Cluster upscaled to 4 nodes

3.3.2 Traffic Splitting and Observability

The goal was to achieve a traffic distribution between the two frontend versions such that v1 receives 75% of the traffic and v2 receives 25%. To do this, the frontend manifest was updated to define two deployments, each labeled with a version tag (v1 and v2), and both matched by the service selector. This allows the service to route traffic to both versions, enabling Istio to split traffic as configured. The use of version labels is key for Istio to distinguish between the two subsets.

```
kubectl apply -f microservices-demo/kustomize/base/frontend.yaml
```

To implement the traffic split, we created two Istio resources: a `DestinationRule` to define the v1 and v2 subsets based on version labels, and a `VirtualService` to control the traffic split between them. The `VirtualService` was configured with a weight of 75 for v1 and 25 for v2. A screenshot is included showing a part of this yaml manifest.

```
kubectl apply -f microservices-demo/istio-manifests/istio-canary.yaml
```

A screenshot of a code editor showing a portion of a Kubernetes manifest file. The file is a VirtualService resource. The code is as follows:

```
15  apiVersion: networking.istio.io/v1beta1
16  kind: VirtualService
17  metadata:
18    name: frontend
19  spec:
20    hosts:
21      - frontend
22    http:
23      - route:
24        - destination:
25          host: frontend
26          subset: v1
27          weight: 75
28        - destination:
29          host: frontend
30          subset: v2
31          weight: 25
```

Figure 3.26: VirtualService manifest.

To monitor and visualize the traffic distribution, we installed Kiali, which provides a graphical interface for observing service mesh traffic and verifying that the canary release is functioning as intended. Because Kiali needs Prometheus to collect metrics, Prometheus was also installed in the `istio-system` namespace. The Kiali dashboard was accessed locally via port forwarding. The following commands were used:

```
kubectl apply -f https://raw.githubusercontent.com/istio/istio/release-1.28/samples/addons/prometheus.yaml
kubectl wait --for=condition=available --timeout=600s deployment\
```

```

/prometheus -n istio-system
kubectl apply -f https://raw.githubusercontent.com/istio/
istio/release-1.20/samples/addons/kiali.yaml
kubectl wait --for=condition=available --timeout=600s deployment\
/kiali -n istio-system
kubectl -n istio-system port-forward svc/kiali 20001:20001

```

In practice, the observed split was close to, but not exactly, the configured weights. This is because Istio decides how to split traffic for each new connection, and some connections may last longer or send more requests than others, so the actual percentages can vary.

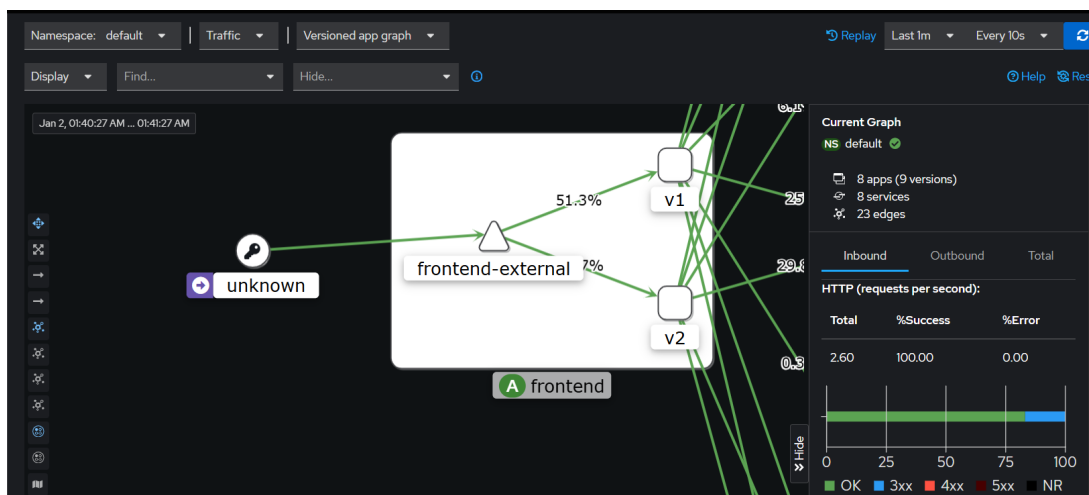


Figure 3.27: Traffic split between frontend v1 and v2 in Kiali

NB: Our initial attempt used the stateless `currencyservice` for canary testing. However, the traffic split was very uneven and unpredictable. This happened because `currencyservice` uses gRPC and HTTP/2, and Istio does not split traffic as expected for these protocols. In particular, requests from the same client or session may always go to the same version, so the traffic distribution can be very different from what was configured.

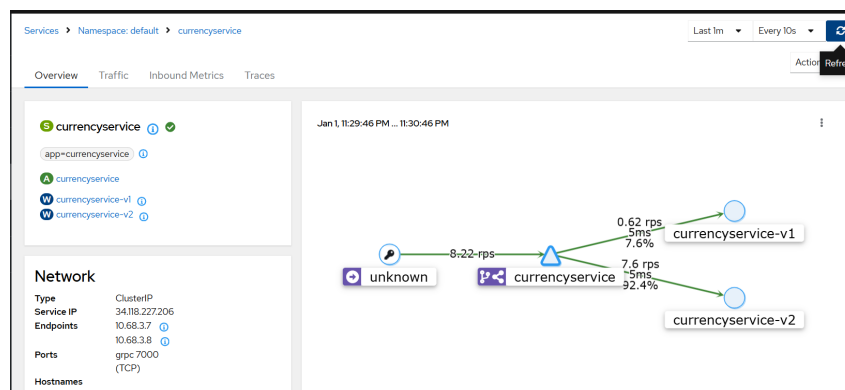


Figure 3.28: Unpredictable traffic split for currencyservice

3.3.3 Full Transition to v2

To complete the rollout, we first modified the VirtualService's configuration to set the weight to 100 for v2 and applied it.

```
apiVersion: networking.istio.io/v1beta1
kind: VirtualService
metadata:
  name: frontend
spec:
  hosts:
  - frontend
  http:
  - route:
    - destination:
        host: frontend
        subset: v2
      weight: 100
```

Figure 3.29: Virtualservice manifest after shift to v2

The result, as observed in Kiali, was a gradual transition of traffic from v1 to v2. This gradual shift happens because some user connections to the frontend stay open for a while, so not all traffic switches to v2 immediately.

As a result, the distribution did not reach a perfect 100% to 0% split right away.

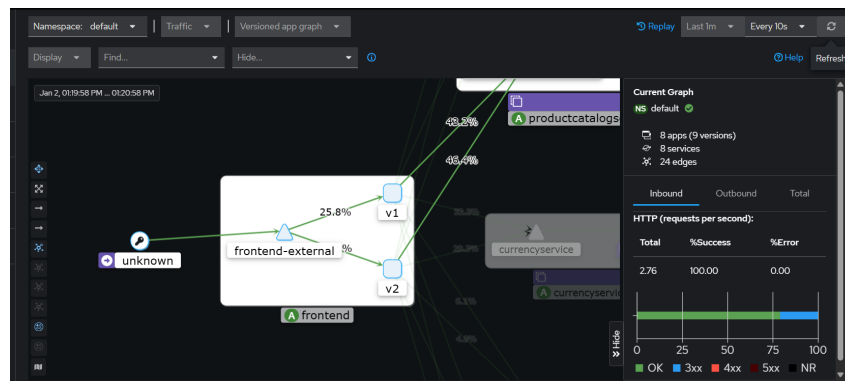


Figure 3.30: Kiali traffic distribution after transition to v2

After the transition, we deleted the v1 deployment, keeping only the v2 version:

```
kubectl delete deployment frontend-v1
```

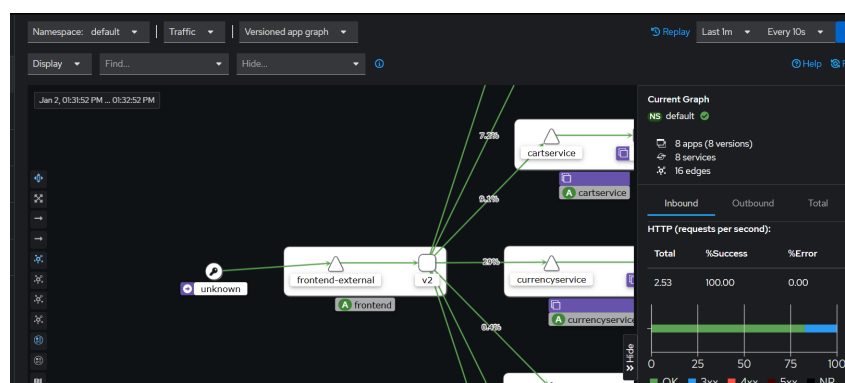


Figure 3.31: Traffic graph after v1 deletion

To confirm the change, we opened the frontend in a browser using the external IP address of the frontend service and observed that the v2 version was indeed being served, as shown in the title **Welcome to Online Boutique (v2)**

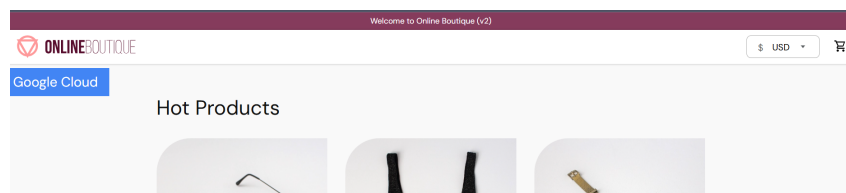


Figure 3.32: Frontend v2 served in browser

Chapter 4

Bonus steps

4.1 Performance evaluation

The performance evaluation clearly reveals that the system reaches a well-defined saturation point, beyond which additional load results in rapidly degrading performance. By correlating the application-level metrics (latency, throughput, failure rate) with the infrastructure-level observations (CPU, memory, and network usage), it is possible to precisely identify the components responsible for this limitation.

The analysis shows that the primary bottlenecks are CPU-bound microservices, specifically:

- `frontend`
- `recommendationservice`
- `productcatalogservice`

These services consistently exhibit the highest CPU utilization during load tests. As concurrency increases, their CPU usage approaches saturation, while other services (such as `emailservice`, `shippingservice`, and `redis`) remain lightly loaded. This confirms that the performance ceiling is not caused by the infrastructure or by backend dependencies, but by computational constraints within these core application components.

The frontend service emerges as a critical bottleneck because it acts as the **main entry point** for user requests and performs request aggregation and orchestration. Similarly, the recommendation service, which involves more complex computation, shows sharp CPU saturation and directly contributes to increased response times. The product catalog service also contributes to the bottleneck due to frequent access and data processing during request handling.

In contrast, memory usage remains stable across all experiments, and network bandwidth is never saturated, eliminating them as limiting factors. The system therefore exhibits a classic CPU-bound performance profile, where throughput is constrained by processing capacity rather than I/O or memory availability.

In summary, the performance ceiling observed in the experiments is caused by CPU saturation in a small subset of critical microservices, rather than by global resource exhaustion. Addressing this bottleneck would require targeted optimizations such as horizontal scaling of the affected services or increasing CPU resource limits.

Bibliography

- [1] Ropars, Thomas. *Terraform Tutorial Cloud Tutorials*. Available at: <https://roparst.gricad-pages.univ-grenoble-alpes.fr/cloud-tutorials/terraform/>
- [2] Helm Authors. *Helm Quickstart Guide*. Available at: <https://helm.sh/docs/intro/quickstart>
- [3] Prometheus Community. *kube-prometheus-stack Helm Charts*. Available at: <https://github.com/prometheus-community/helm-charts>
- [4] Prometheus Community. *GitHub Issue: kube-prometheus-stack admission webhook problem*. Available at: <https://github.com/prometheus-community/helm-charts/issues/418>
- [5] Spacelift. *How to Set Up Prometheus Monitoring on a Kubernetes Cluster*. Available at: <https://spacelift.io/blog/prometheus-kubernetes#how-to-set-up-prometheus-monitoring-on-a-kubernetes-cluster>